

Arquitetura de BI + Dashboard — crm_inquilino_0040_infratecnica_v1

Documento técnico que descreve a **arquitetura** da plataforma de BI Comercial do cliente Dommus #0040 — Infratécnica: estrutura do ETL, modelo de dados, dashboard e segurança.

Cliente	Dommus #0040 — Infratécnica
Tipo de projeto	BI Comercial (ETL + Dashboard)
Modelagem dimensional	Star schema (1 fato central por assunto + dimensões compartilhadas)
Repositório	github.com/klingerlima23-hub/dommus-bi-infratecnica
URL pública	https://dommus-bi-infratecnica.vercel.app

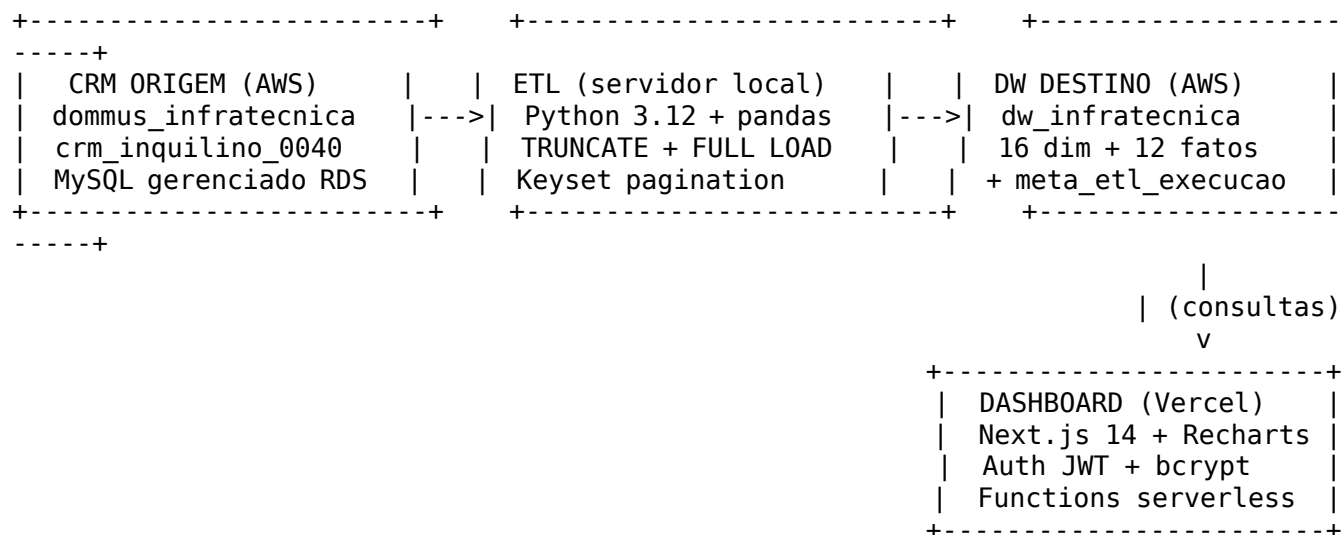
Última atualização: **28/05/2026** por **Klinger Lima**, Engenheiro de Dados.

Sumário

1. [Visão geral da arquitetura](#)
 2. [ETL — origem, destino, processo](#)
 3. [Modelo de dados — dw_infratecnica](#)
 4. [Dashboard — Next.js em ambiente cloud serverless](#)
 5. [Segurança e gestão de credenciais](#)
 6. [Separação de ambientes](#)
 7. [Solicitações — histórico de alterações](#)
-

1. Visão geral da arquitetura

A solução é dividida em três ambientes independentes, cada um com sua própria responsabilidade e ciclo de vida:



Fluxo dos dados: 1. O **CRM origem** (na AWS) recebe os dados operacionais do cliente Infratécnica. 2. O **ETL** (rodando no servidor local Dommus) lê os dados do CRM, transforma e carrega no DW destino (também na AWS). 3. O **DW destino** (dw_infratecnica) é onde os dados ficam consolidados em modelo estrela, prontos para consulta analítica. 4. O **Dashboard** (Next.js, hospedado na Vercel) consulta o DW destino via API serverless e renderiza KPIs e gráficos.

2. ETL — origem, destino, processo

2.1. Bancos de origem

Banco	Conteúdo	Localização
dommus_infratecnica	Dados detalhados de cliente, usuário, equipe, gerente, corretor, empreendimento, processo de venda	MySQL gerenciado AWS RDS
crm_inquilino_0040	Oportunidades, leads, funil, mídia, campanha, origem, status, etapa	MySQL gerenciado AWS RDS

2.2. Banco de destino

Banco	Localização	Schema
dw_infratecnica	MySQL na AWS (18.209.74.87:3310)	16 dimensões + 12 fatos + meta_etl_execucao = 29 tabelas, ~410 colunas

2.3. Onde o ETL roda

O ETL é **executado no servidor local Dommus** (Python 3.12, sem acesso público), no IP de extração liberado pelo CRM origem. Esse servidor estabelece conexões seguras para os bancos de

origem (CRMs) e destino (dw_infratecnica) na AWS. As credenciais ficam fora do código-fonte (ver Seção 5).

2.4. Estratégia de carga

TRUNCATE + FULL LOAD com paginação adaptativa:

Etapa	Estratégia
Para cada tabela do DW	TRUNCATE TABLE antes de inserir
Leitura da origem	Keyset pagination (page-size adaptativa: 5000 → 1000 → 500)
Escrita no DW	INSERT em lote (chunksize 500) com retry e backoff exponencial
Falhas de conexão	Mitigação 6 camadas: use_pure=True, SESSION_INIT, pool_recycle, retry com backoff, page-size adaptativa, guard Python 3.12
Orquestração	Um processo Python por tabela (subprocess isolado)
Auditoria	Cada execução registra iniciado_em, concluido_em (UTC) e status em meta_etl_execucao

2.5. Tempo estimado de carga

Para volumes típicos (alguns milhares de vendas, dezenas de milhares de leads):

Categoria	Tempo aproximado
Dimensões pequenas (até 100 linhas)	~12 s cada
Dimensões médias (100 a 10k)	~12 a 25 s cada
Fatos médios	~12 a 30 s cada
Fato grande (f_venda, f_espelho_de_venda)	~35 s
Total típico	~7 a 10 minutos

Volumes maiores escalam linearmente devido à paginação adaptativa.

2.6. Indicador “Atualizado em” no dashboard

O Sidebar exibe a data/hora da última execução bem-sucedida do ETL. O endpoint /api/data/etl-status lê a coluna concluido_em (gravada em UTC) da tabela meta_etl_execucao onde status='ok', e o navegador converte automaticamente para o fuso local do usuário.

3. Modelo de dados — dw_infratecnica

3.1. Modelagem dimensional (Star Schema)

Cada **fato** representa um evento de negócio (lead, atendimento, visita, venda, descarte) e referencia múltiplas **dimensões** (corretor, empreendimento, campanha, mídia etc.) por suas chaves naturais.

3.2. Inventário de dimensões (16 tabelas)

d_cliente_dommus, d_funil, d_etapa_funil, d_fase_funil, d_status_funil, d_etapa_oportunidade, d_fase_etapa, d_origem, d_midia, d_campanha, d_empreendimento, d_equipe, d_corretor, d_gerente, d_usuario, d_usuario_gu.

3.3. Inventário de fatos (12 tabelas)

f_lead, f_oportunidade, f_atendimento_oportunidade, f_visita, f_descarte, f_funil, f_investimento, f_evolucao_processo, f_historico_etapa_processo, f_unidade, f_espelho_de_venda, f_venda (esta com 121 colunas — proponentes 1/2/3, processo, análise de crédito, banco, venda contabilizada).

3.4. Tabela de auditoria

meta_etl_execucao — registra cada execução do orquestrador (id, iniciado_em UTC, concluido_em UTC, status, contagens de scripts executados e pulados).

3.5. Tipos de dados

Os tipos são derivados dos comentários -- tipo em cada consulta_*.sql:

Tipo no comentário	Tipo MySQL gerado
-- int	BIGINT NULL (suporta IDs grandes do CRM)
-- varchar(N)	VARCHAR(N) NULL
-- date	DATE NULL
-- datetime	DATETIME NULL
-- decimal(M,N)	DECIMAL(M,N) NULL
-- text	TEXT NULL

Cada tabela tem KEY idx_<tabela>_<id> na PK natural para acelerar consultas do dashboard.

3.6. Charset e collation

Todas as tabelas usam utf8mb4 / utf8mb4_unicode_ci (suporte completo a acentos e emojis em nomes de empreendimentos, descrições etc.).

4. Dashboard — Next.js em ambiente cloud serverless

4.1. Tecnologia

Camada	Tecnologia	Versão
Framework	Next.js (App Router)	14.2.15
Linguagem	TypeScript	5.6.x
UI	Tailwind CSS + Radix UI	3.4.x
Gráficos	Recharts	2.13.x
Auth (Edge)	jose (JWT verify em middleware)	5.9.x
Auth (Node)	bcryptjs	2.4.x
MySQL client	mysql2	3.11.x
Hospedagem	Vercel (cloud serverless)	—

4.2. Estrutura

- **Rotas públicas:** /login
- **Rotas protegidas (middleware):** /dashboard/*, /api/data/*
- **Páginas:**
 - /dashboard/vendas (sub-abas Visão Atual / Funil de Venda — 5 etapas)
 - /dashboard/estoque
 - /dashboard/visitas
 - /dashboard/lead (sub-abas Visão Geral / Funil de Investimento)
- **APIs de autenticação:** /api/auth/login, /api/auth/logout
- **APIs de dados:**
 - /api/data/vendas
 - /api/data/funil-venda
 - /api/data/estoque
 - /api/data/visitas
 - /api/data/oportunidade
 - /api/data/funil
 - /api/data/cliente
 - /api/data/etl-status

4.3. Conexão com o DW

Cada chamada a /api/data/* abre conexão MySQL via pool (mysql2) com o dw_infratecnica. O pool é guardado em globalThis para reuso dentro da mesma function serverless.

4.4. Identidade visual

O Sidebar carrega logo e nome do cliente dinamicamente da tabela d_cliente_dommus (via /api/data/cliente). A logo “Grupo Infratécnica” é fornecida pelo próprio cliente e armazenada no DW.

5. Segurança e gestão de credenciais

5.1. Princípios

1. **Credenciais nunca ficam no código.** Senhas, tokens e secrets ficam em arquivos não versionados (.env.local, .streamlit/secrets.toml) que estão no .gitignore.
2. **Separação por ambiente.** Dev/local usa arquivos próprios; produção (Vercel) usa Environment Variables encriptadas.
3. **Princípio do menor privilégio.** O dashboard só faz SELECT no DW.
4. **Senhas de usuário com hash.** Tabela d_usuario_gu armazena hashes bcrypt (custo 10).
5. **JWT com expiração.** Cookie dommus_session é httpOnly, secure (em prod), TTL 24 h, assinado HS256 com JWT_SECRET aleatório de 64 bytes.

5.2. Onde ficam as credenciais

Credencial	Local (DEV)	Local (PRODUÇÃO)
Senha do MySQL DW	.env.local (não commitado)	Environment Variables na Vercel
JWT_SECRET	.env.local	Environment Variables na Vercel
Credenciais do CRM origem (ETL)	etl/pipeline/.streamlit/secrets.toml (não commitado)	NA (ETL toda só local)

5.3. Proteção do código-fonte

- Repositório GitHub **privado**: github.com/klingerlima23-hub/dommus-bi-infratecnica
- .gitignore cobre: node_modules/, .next/, .env*, **/secrets.toml, __pycache__/, *.pyc, .pipeline_state.json

5.4. Segurança do banco de dados

- O DW MySQL na AWS (18.209.74.87:3310) está atrás de um Security Group que aceita conexões apenas de:
 1. IPs específicos da rede Dommus (para o ETL local)
 2. IPs de egress da Vercel (para o dashboard em produção)
 - Nenhum acesso 0.0.0.0/0 (banco não é público).
-

6. Separação de ambientes

Componente	Onde roda	Acessa
ETL	Servidor local Dommus (Python 3.12)	CRM origem (RDS) + DW destino (RDS)
DW	AWS RDS MySQL	(só recebe escritas do ETL e leituras do dashboard)
Dashboard	Vercel (Hobby plan)	DW destino (RDS)

Implicação prática: - O ETL **não depende** do dashboard estar no ar - O dashboard **não depende** do ETL estar rodando - Falha em qualquer um dos três não derruba os outros

7. Solicitações — histórico de alterações

Esta seção lista todas as solicitações de mudança feitas pelo cliente (Infratécnica) ou pelo time Dommus que impactaram o comportamento do produto. Cada entrada registra a data, o autor, o pedido original e o que foi implementado.

CR-001 — Migração do dashboard Streamlit para Next.js

Campo	Valor
Data	27/05/2026
Solicitante	Dommus (interno)
Tipo	Refatoração arquitetural

Pedido: modernizar o dashboard Infratécnica, substituindo o Streamlit (v001) por Next.js + Tailwind + Recharts, mantendo a lógica de negócio idêntica e seguindo o padrão da BNR V1.

Implementação: - Cópia dos componentes, paleta, layout e páginas do projeto BNR V1. - Logo Infratécnica e identidade visual preservadas (public/logo/logo_infratecnica.png). - APIs src/app/api/data/* e queries SQL mantidas com adaptações para o modelo Infratécnica. - Endpoint /api/data/etl-status adicionado para suportar o indicador “Atualizado em” no Sidebar.

CR-002 — Pipeline de auditoria do ETL

Campo	Valor
Data	27/05/2026
Solicitante	Dommus (interno)
Tipo	Funcionalidade

Pedido: o Sidebar exibia “Atualizado em —” porque a tabela meta_etl_execucao não existia e o pipeline não registrava execuções.

Implementação: - Adicionada a DDL da meta_etl_execucao em etl/pipeline/criar_dw_infratecnica.sql. - Adicionadas as funções _meta_etl_ensure_table, _meta_etl_inicio e _meta_etl_fim no orquestrador etl_inquilino_0040_infratecnica_executa_pipeline.py. - Cada execução do ETL agora registra início e fim em UTC, e o Sidebar mostra o timestamp convertido para o fuso do navegador.

CR-003 — Card “VENDIDA” na aba Estoque

Campo	Valor
Data	28/05/2026
Solicitante	Camila (cliente Infratécnica)
Tipo	Regra de negócio

Pedido (literal): “VENDIDA é toda e qualquer unidade que estiver nos quadros Vermelho, Azul, Registro (roxo) e esse (verdinho) = vendida (cliente não registrado). Pode corrigir?”

Mapeamento de códigos:

Cor	Código	Descrição
Vermelho	V	VENDIDA
Azul	A	ASSINADA NO BANCO
Roxo	S	REGISTRADA
Verde claro	X	VENDIDA (cliente não registrado)
Verde escuro	D	DISPONÍVEL PARA VENDA
—	R	RESERVADA
—	B	RESERVA TÉCNICA
—	I	INDISPONÍVEL PARA VENDA

Implementação: - src/app/api/data/estoque/route.ts: adicionada a coluna disponibilidade (código de uma letra) no SELECT. - src/app/dashboard/estoque/page.tsx: função classificarStatus() reescrita para classificar pelo código (mais robusto): - V, X, A, S → **VENDIDA** - D → **DISPONIVEL** - R, B → **RESERVA** - demais (I e outros) → **OUTROS** - O card “Registrada” foi mantido na interface (mostra 0 — todas as S foram migradas para VENDIDA). - A mesma classificação se aplica ao gráfico de pizza “Distribuição por Status”.

CR-004 — Bug: card VENDA zerado na aba Vendas → Funil de Venda

Campo	Valor
Data	28/05/2026
Solicitante	Klinger Lima (interno)
Tipo	Bug

Pedido: o card “VENDA” do Funil de Venda exibia 0 mesmo com etapas anteriores (Cadastro 82, Pastas 22, Aprovado IF 32, Contrato 18) não zeradas.

Causa raiz: a API /api/data/funil-venda ignorava a coluna real venda_contabilizado_em da tabela f_venda (populada pelo ETL via tb_venda.contabilizado_em do CRM). Em vez disso, inferia a venda pela flag processo_unidade_id > 0 AND processo_data_venda IS NOT NULL, regra que não casava no modelo de dados Infratécnica.

Implementação: - src/app/api/data/funil-venda/route.ts: passou a selecionar v.venda_contabilizado_em diretamente da f_venda. - A flag is_venda agora é derivada como CASE WHEN v.venda_contabilizado_em IS NOT NULL THEN 1 ELSE 0 END.

CR-005 — JOIN com d_campanha em /api/data/oportunidade

Campo	Valor
Data	28/05/2026
Solicitante	Klinger Lima (interno)
Tipo	Funcionalidade

Pedido: o gráfico de rosca “Oportunidades por Campanhas” da aba Lead mostrava “Sem campanha” para 100% das oportunidades.

Implementação: - src/app/api/data/oportunidade/route.ts: adicionado LEFT JOIN d_campanha cmp ON o.id_campanha = cmp.id_campanha direto (sem necessidade do passo intermediário via f_lead que o BNR usa, porque o modelo Infratécnica já tem id_campanha denormalizado em ‘f_opor